Industrial Internship Report on

”Grocery Delivery Application”

Prepared by

Isha Sharma

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 4 weeks’ time. My project was a full-stack Grocery Delivery Application, covering catalog browsing, cart management, delivery slot selection, payment, and order tracking, built using React, Spring Boot, and PostgreSQL. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

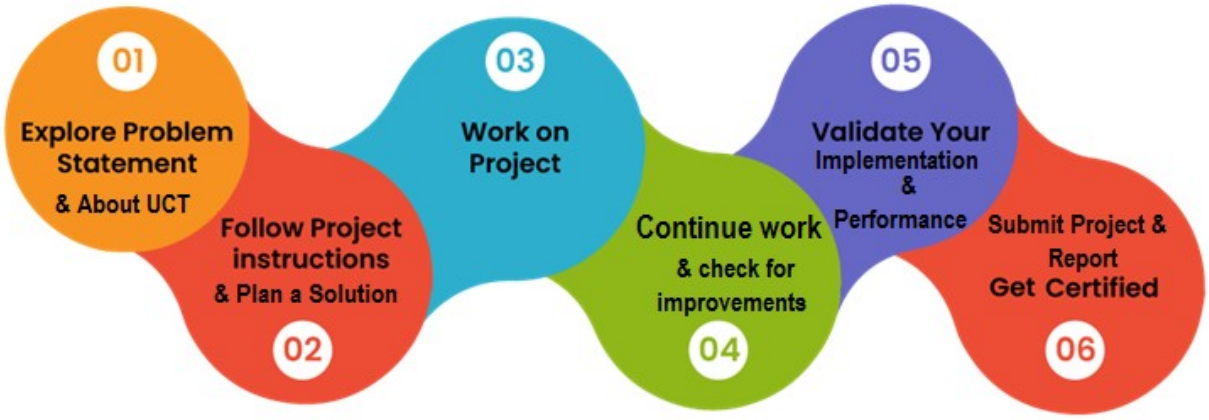
Over four weeks, I designed and built a full-stack Grocery Delivery Application, moving from requirement scoping and system design in Week 1, through core commerce features (cart, delivery slots, order flow) in Week 2, payment and order tracking in Week 3, to polish, documentation, and final review preparation in Week 4.

An internship like this is valuable in career development because it forces you to work with an ambiguous, real-world problem statement under a fixed deadline, rather than a clearly defined academic assignment, which builds skills in scoping, prioritization, and independent decision-making that the classroom does not.

My project was a Grocery Delivery Application: a platform allowing users to browse a grocery catalog, manage a cart, select a delivery slot, complete payment, and track their order in real time, built on a React frontend, Spring Boot backend, and PostgreSQL database.

This internship, offered through upskill Campus and The IoT Academy in collaboration with UCT, gave me the opportunity to work on an industry-style project end-to-end, from scoping through delivery.

The program was planned across four weeks, with each week building on the previous one: Week 1 covered scoping and system design, Week 2 covered core commerce features, Week 3 covered payments and order tracking, and Week 4 covered polish and final documentation.



This internship taught me how to take an ambiguous, mixed-domain brief and scope it into a realistic MVP under a hard deadline, how to make and document database and architecture trade-offs, and how to handle concurrency and reliability issues (stock validation, payment callbacks) that only surface once a system is used end-to-end. Overall, it was a great experience that bridged the gap between academic project work and real industry practice.

I would like to thank the mentors and coordinators at upskill Campus, The IoT Academy, and UniConverge Technologies Pvt Ltd for their guidance and support throughout this internship.

To my juniors and peers: take the time to properly scope your problem statement before writing any code — the hours spent clarifying what to build and what to skip pay off many times over during implementation.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

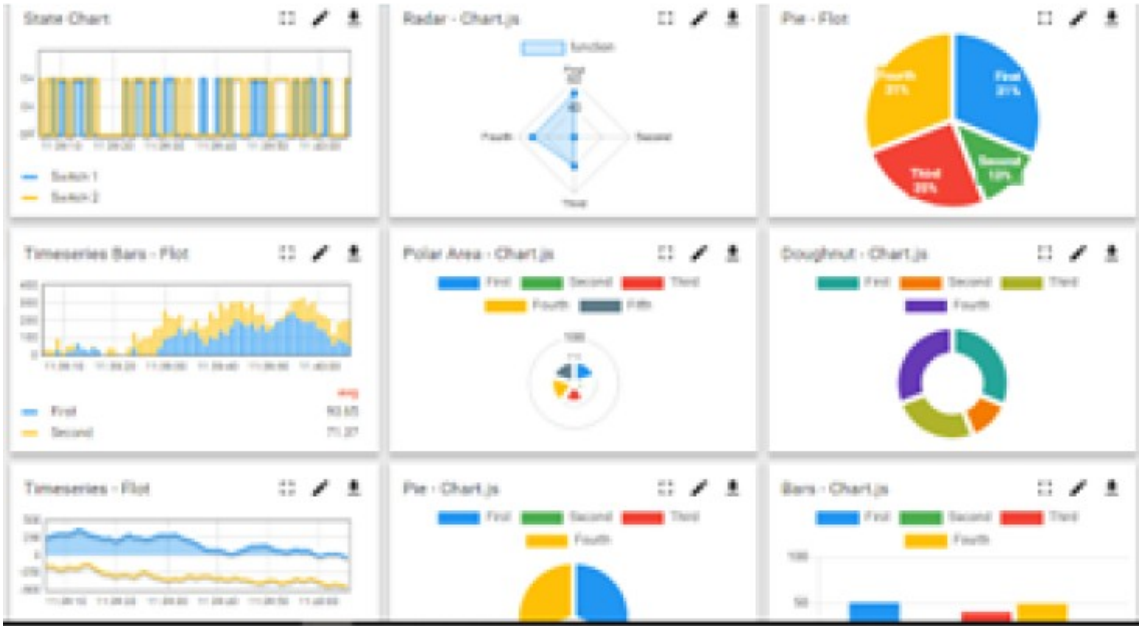
i. UCT IoT Platform (uct Insight)

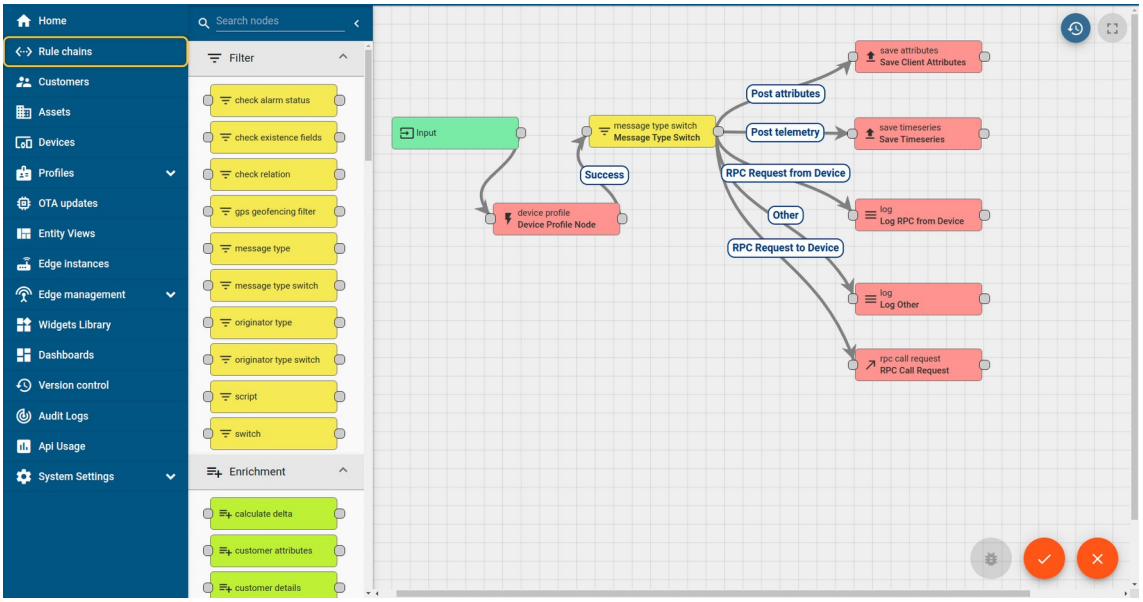
UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

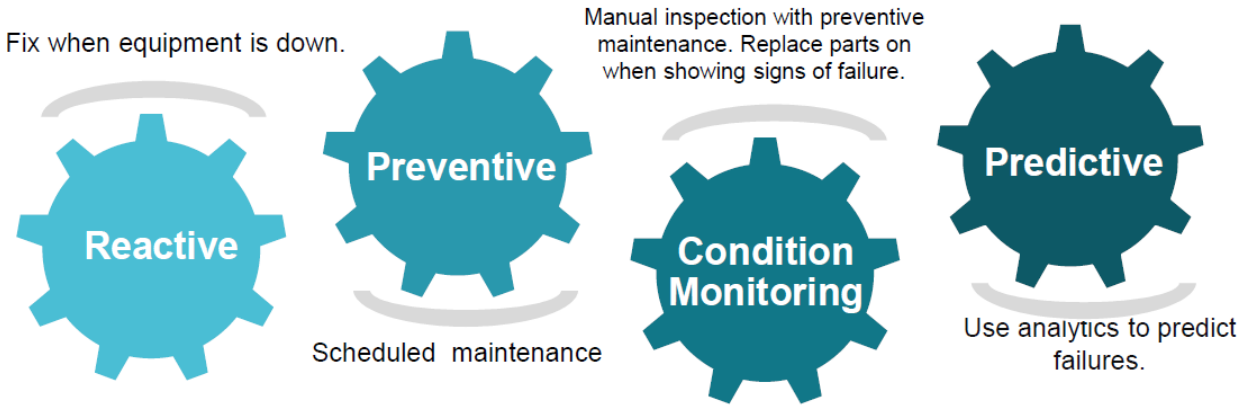
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

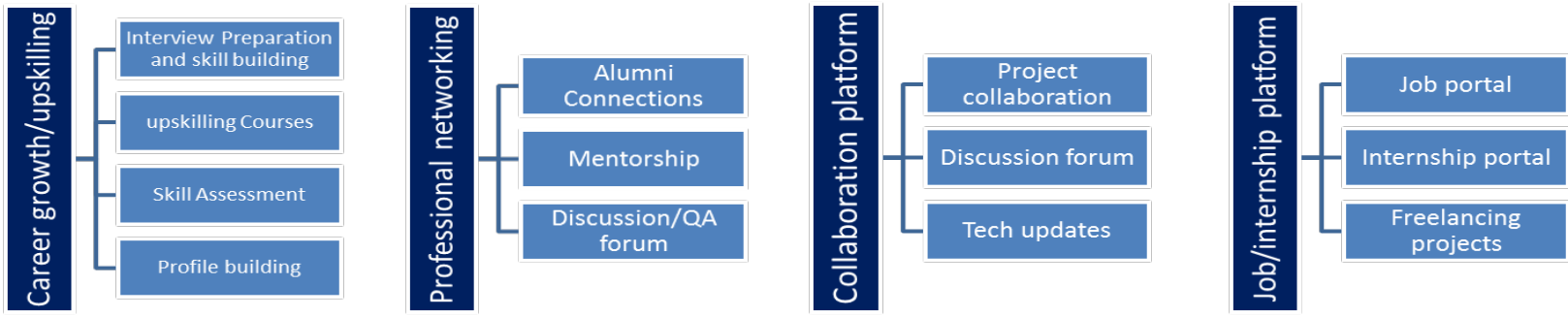
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] React Documentation — https://react.dev

[2] Spring Boot Reference Documentation — https://spring.io/projects/spring-boot

[3] PostgreSQL Documentation — https://www.postgresql.org/docs/

2.6 Glossary

Terms	Acronym
MVP	Minimum Viable Product
API	Application Programming Interface
JWT	JSON Web Token
REST	Representational State Transfer
ER (Diagram)	Entity-Relationship (Diagram)

3 Problem Statement

In the assigned problem statement, the brief mixed requirements from three different delivery domains — grocery, automotive parts, and restaurant delivery — without clearly separating which features belonged to which.

The task was to scope this down into a realistic, buildable grocery delivery application within 4 weeks: a platform where a user can browse a grocery catalog, add items to a cart, choose a delivery slot, pay, and track the order until delivery — while excluding out-of-scope features like vehicle fitment lookup or restaurant menu management that did not apply to grocery delivery.

4 Existing and Proposed solution

Existing grocery delivery platforms (e.g. BigBasket, Blinkit, Zepto) already solve catalog browsing, cart, and delivery at scale, but as large commercial platforms they are closed systems — their handling of stock/slot concurrency, payment reliability, and order-history accuracy is not visible or reusable for a learning project, and their scope (dark stores, hyperlocal logistics) is far beyond a 4-week build.

The proposed solution is a self-contained Grocery Delivery Application covering the core commerce flow end-to-end: catalog browsing, cart with live stock validation, delivery slot selection, an order draft/review step, payment with a fallback for unreliable gateway callbacks, and order tracking through to delivery — built on a React frontend, Spring Boot backend, and PostgreSQL database.

The value addition is in the engineering detail applied to a deliberately small scope: concurrency-safe stock and slot validation, an order schema that preserves item-wise price at the time of purchase, and a payment status fallback — details that are easy to skip in a rushed academic project but are exactly what make a commerce system trustworthy in production.

4.1 Code submission (Github link)

https://github.com/caspian-1901/upskillCampus.git

4.2 Report submission (Github link) :

https://github.com/caspian-1901/upskillCampus.git

4.3

5 Proposed Design/ Model

The system follows a three-tier architecture: a React + Tailwind CSS frontend, a Spring Boot backend exposing REST APIs, and a PostgreSQL database. The flow starts with authentication (sign-up/login), moves through catalog browsing, cart management with stock validation, delivery slot selection, and an order draft/review step, then to payment and finally order tracking through to delivery.

5.1 High Level Diagram (if applicable)

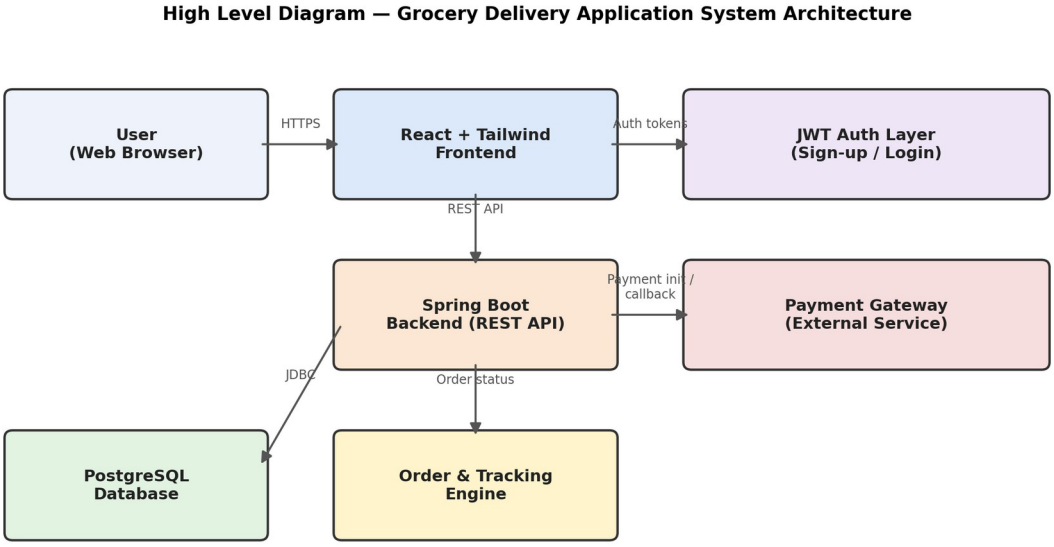
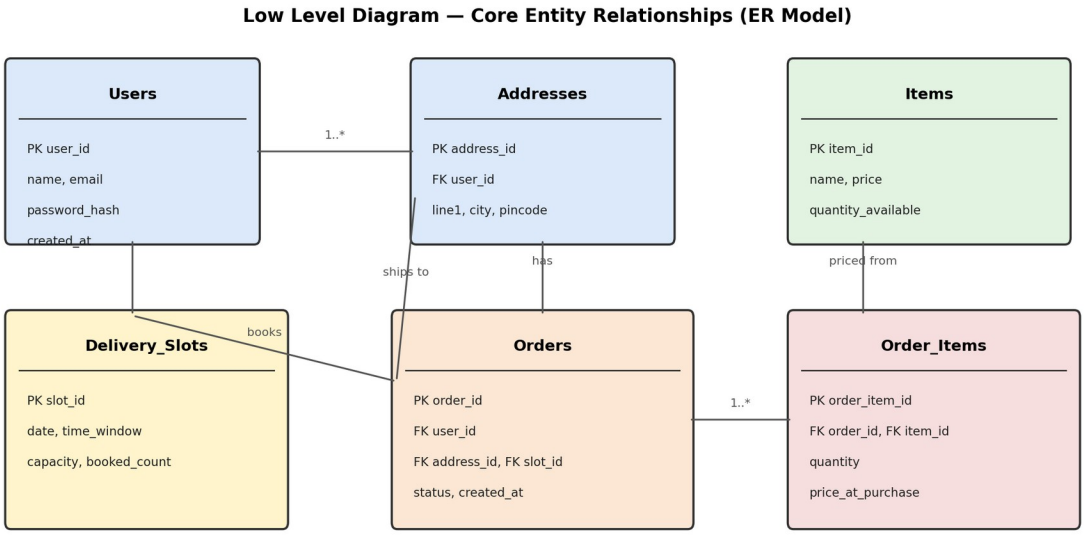


Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)



5.3 Interfaces (if applicable)

Key interfaces: a REST API between the React frontend and Spring Boot backend (JSON over HTTPS, JWT-authenticated), a JDBC connection between the backend and PostgreSQL, and a webhook/callback interface with the external payment gateway, backed by a polling fallback for delayed callbacks.

6 Performance Test

This section explains the constraints considered and how the design was validated against real usage scenarios rather than a purely academic build.

The main constraints identified were: (1) data consistency under concurrent cart and slot updates, (2) reliability of external payment gateway callbacks, and (3) query performance for order history with accurate historical pricing.

Concurrency was handled by re-validating stock and slot availability server-side at the point of write rather than trusting client state. Payment reliability was handled with a fallback that queries the gateway directly if no callback arrives within a set window. Order history performance was addressed by storing item-wise price at the time of purchase in the Order_Items table, avoiding costly joins against a mutable catalog price.

Testing against sample queries such as "show order history with prices at time of purchase" confirmed the schema returned correct historical prices without extra joins, and concurrent cart-update tests confirmed stock could not be oversold.

These constraints centered on data consistency and reliability rather than raw performance metrics like memory or throughput, given the project's 4-week academic scope.

Load and throughput testing under production-scale traffic was not performed given the project timeline; this is flagged as a recommended next step before any real deployment.

6.1 Test Plan/ Test Cases

Test cases covered: sign-up/login, cart add/remove/update with stock limits, slot booking with concurrent users, order draft creation with missing address/slot, payment success and failure paths, and order status transitions from Placed through Delivered.

6.2 Test Procedure

Each module was tested independently after implementation, followed by end-to-end integration testing of the complete flow (sign-up through order tracking) once payment and tracking were integrated in Week 3. Concurrent update scenarios were tested by simulating simultaneous cart/slot updates from multiple sessions.

6.3 Performance Outcome

All test cases passed after fixes: stock could no longer be oversold under concurrent updates, stale slot availability no longer appeared at final review, and orders no longer got stuck in a pending state when payment callbacks were delayed, thanks to the status-check fallback.

7 My learnings

This internship taught me how to convert an ambiguous, mixed-domain brief into a realistic, scoped MVP under a hard deadline, and how to make and justify database and architecture trade-offs rather than defaulting to the most "complete" design. I also learned that reliability issues — race conditions on stock/slots, unreliable payment callbacks — only surface once a system is tested end-to-end, which reinforced the value of integration testing over purely unit-level testing. These are directly transferable to real engineering roles, where requirements are rarely fully specified and production reliability matters as much as feature completeness.

8 Future work scope

Ideas deferred due to the 4-week timeline: personalized item recommendations based on order history, a loyalty/rewards program, an admin dashboard for inventory and order management, push notifications for order status changes, and load/throughput testing under production-scale traffic before any real deployment.